Gem Detector — Build Handoff

Nov 3, 2026 · @Sean Cantrell

Goal

Build an app that watches NYC rental listings on a schedule and alerts Geo and Arushi within hours when a listing is a "gem." Phase 1 serves the two of us. Nothing in the design should block opening it to the public later.

A gem scores well on four parts. Each part is scored separately so every alert shows why it fired:

- **Floor-plan fit:** room dimensions and layout pass our hard constraints (kitchen space, a wall for the computer setup and game systems, room for a Murphy bed) and score well on soft ones.
- **Price:** net effective rent sits well below what a model of comparable units predicts. Pricing below comparables is our working definition of a non-gouging landlord, and it uses listing data only.
- **Commute:** door-to-door transit time to both offices at real departure times.
- **Neighborhood:** inside our target list, weighted by preference.

Out of scope for now: price-history scoring, ingesting alert emails, and sources other than StreetEasy.

Hard constraints

These are fixed. If the builder thinks one blocks progress, it raises that with Geo rather than working around it.

1. **No channel that requires contacting anyone.** That rules out licensed feeds (REBNY RLS, IDX/VOW), broker partnerships, and any API that needs an application or approval.
2. **Fully automated.** No human in the loop and no driving Geo's own browser. Browser tools are acceptable only inside an agentic workflow that runs unattended.
3. **Pages are fetched by an agent.** StreetEasy pages are retrieved by an LLM agent through its web-fetch tool, which respects robots.txt. In prototyping, a plain shell request to StreetEasy that set a browser user agent was blocked as bot-check circumvention.

4. **No bot-check circumvention of any kind.** No spoofed user agents, rotating proxies, CAPTCHA solvers or stealth headless browsers. Do not "fix" a block with any of these.
5. **Plain HTTP for everything else.** Listing images on `photos.zillowstatic.com`, StreetEasy's public CSVs, and MTA transit files all download with ordinary requests.
6. **Polite volume.** Only the target neighborhoods and filters, a few runs a day, sequential fetches.
7. **Deterministic code wherever possible.** The LLM fetches pages and reads floor-plan images. Parsing, storage, scoring and alerting are ordinary, tested code.

Verified data access

Every row below was tested on 2026-11-03. Pages were fetched by the agent's web-fetch tool, and files were downloaded with plain `curl`.

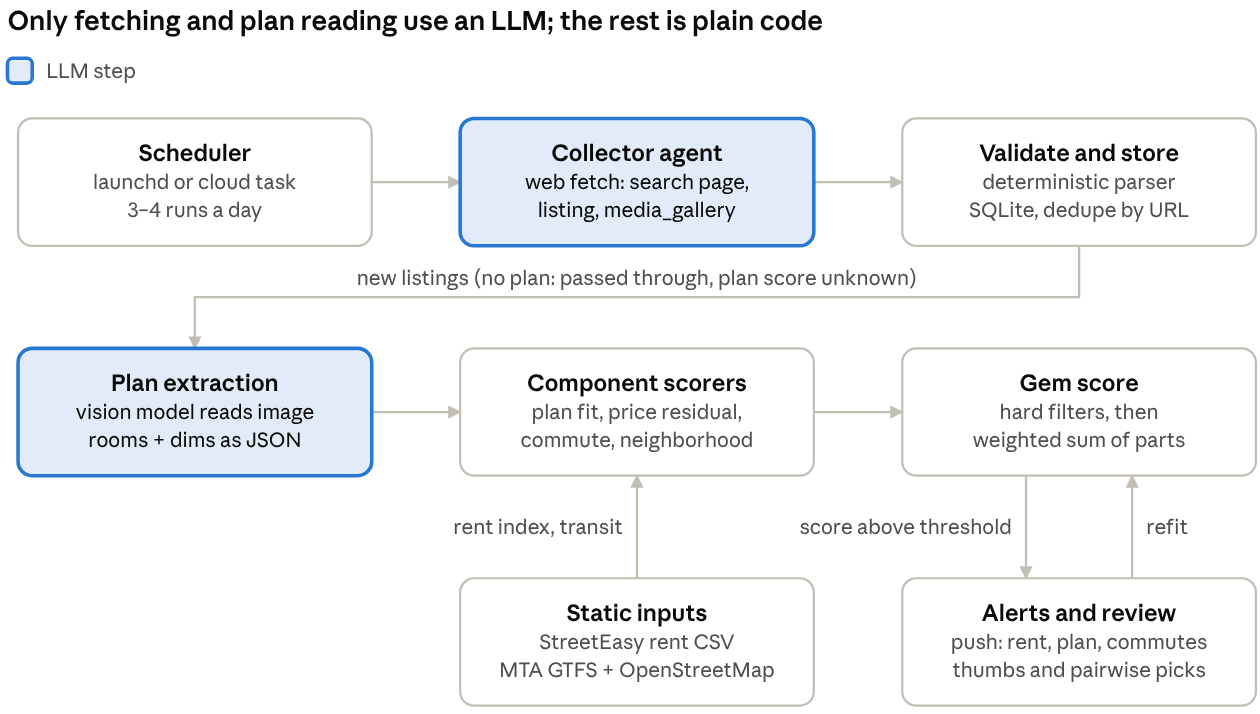
Source	URL pattern	What it yields	Status
Search page	`streeteasy.com/for-rent/<neighborhood-slug>/beds:2`	Listing cards: address, unit, base rent, net effective rent, free months, lease term, beds/baths, sq ft (sometimes), listing URL. No listed date.	Works (agent fetch)
Listing page	`streeteasy.com/building/<building-slug>/<unit>`	Price, beds/baths, sq ft, listed date, days on market, broker, building facts (units, stories, year), amenities, unit features, gallery image URLs, and a "Floor plan" label when a plan exists	Works (agent fetch)
Media gallery	`<listing URL>/media_gallery`	Every image, labelled: floor plans have alt text `floor plan 1`, `floor plan 2`, …; photos have `photo N`	Works (agent fetch). **This is where floor plans are found.**
Images	`photos.zillowstatic.com/fp/<hash>-full.webp` (about 1000 px) or `-se_extra_large_1500_800.webp`	Floor-plan and photo files	Works (plain `curl`)
Rent index	`streeteasy-market-data-download.s3.amazonaws.com/rentals/TwoBd/medianAskingRent_TwoBd.zip` (also `rentals/All/medianAskingRent_All.zip`)	Monthly median asking rent by neighborhood and borough, CSV, 2010 to present	Works (plain `curl`). Note the path segment is `TwoBd`; `2Bd` returns 403.
Transit timetables	`rrgtfsfeeds.s3.amazonaws.com/gtfs_subway.zip`, `gtfs_b.zip` (Brooklyn bus)	MTA GTFS schedules (GTFS is the standard transit timetable format)	Works (plain `curl`). The other borough bus files are presumably named the same way; not tested.

Source	URL pattern	What it yields	Status
Street network	OpenStreetMap New York extract (e.g. Geofabrik)	Walking legs for commute routing	Blocked in the Claude cloud sandbox. Should work on Geo's machine.
Old-style listing URLs	`streeteasy.com/rental/<id>`	Nothing	Refused by the fetch tool (disallowed by robots.txt). Use `/building/<slug>/<unit>` URLs instead.

Worked example (floor-plan path). For The Parkline #22G, the listing page showed a "Floor plan" label. `/building/the-parkline/22g/media_gallery` gave the plan's image hash with alt text `floor plan 1`. `photos.zillowstatic.com/fp/7c8acf8699c765b73c647d470033757e-full.webp` downloaded at 1000×773. The plan is a broker drawing with dimensions printed on it (bedrooms 11′4″ × 10′0″ and 11′4″ × 9′11″, living/dining 15′4″ × 12′5″), so a vision model can read room sizes directly. Use it as the first golden test case.

Coverage. In a 3-listing sample, only 1 had a floor plan. Expect a large share of listings with no plan; the pipeline must handle that case (see Models).

Architecture



gem detector pipeline · 8 components, 2 LLM steps

The agent's only output is JSON. Validation, storage, scoring and alerting are deterministic code, and the review picks refit the score weights every week.

Models

Notation: n and m index listings; Greek letters index feature or score-component axes; repeated indices are summed (Einstein convention).

Floor-plan extraction

A vision model reads each plan image into a fixed JSON schema: rooms (type, label, printed dimensions, area), bathroom count, washer/dryer, dishwasher, closets, windows and exposure per room, stated total sq ft, and a confidence per field.

- **Printed dimensions:** most broker plans print them, as in the Parkline example. Use them first.
- **No printed dimensions:** set the scale from standard objects (doors about 30–36 in, bathtubs about 60 in, ranges about 30 in, refrigerators about 30–36 in), then measure rooms in pixels.
- **Constraint checks** run in code on the extracted geometry: a clear wall and floor depth for the Murphy bed, a wall length for the desk and game setup, and a minimum kitchen counter run. Geo supplies the numbers (see Open decisions).
- **No plan:** fall back to the listing's stated sq ft and room count, and mark the plan score unknown rather than zero. Don't estimate scale from photo furniture: many listings are virtually staged, meaning the furniture is computer-rendered.

Commute

Use r5py, a Python wrapper around the R5 transit router, with MTA GTFS files plus an OpenStreetMap street network. It computes door-to-door transit times from each listing to each office.

- Take the median over a weekday departure window (default 8:00–9:00). Medians over a window are more robust than one departure time.
- Offices: O_1 = Radical AI (Brooklyn Navy Yard) and O_2 = Arushi's office (address TBD).
- With t_{n_1} and t_{n_2} as listing n's times to the two offices, the commute cost is a choice for Geo: the max of the two (worst-off commuter) or a weighted sum.

Price: a hedonic residual

A hedonic model predicts a unit's rent from its features. Fit a ridge regression (least squares with an L2 penalty on the coefficients) of log net effective rent on a feature vector x_{na}: neighborhood indicators, beds, baths, sq ft from the plan or listing, building age, doorman, elevator, in-unit laundry, outdoor space and floor.

$$\log r^{\mathrm{net}}_{n} = x_{n\alpha}\,\beta_{\alpha} + \varepsilon_{n}, \qquad r^{\mathrm{net}}_{n} = r^{\mathrm{base}}_{n}\left(1 - \frac{f_{n}}{T_{n}}\right) + \phi_{n}$$

Here f_n is free months, T_n is lease length in months, and ϕ_n is any mandatory monthly fee. The price score is the negative residual, $-e_n = -(\log r_n - x_{na}\,\hat{\beta}_a)$. A large positive value means the unit rents well below comparable units.

Until roughly 300 listings have been collected, anchor the neighborhood terms to StreetEasy's monthly median CSV, smoothed with a 3-month mean because single months are noisy.

Gem score

Each listing gets a component vector $s_{n\gamma}$ over $\gamma \in \{\text{plan, price, commute, neighborhood}\}$, standardized across the current pool. Hard filters $H_n \in \{0, 1\}$ come first, and the score is a weighted sum:

$$S_{n} = H_{n}\, s_{n\gamma}\, w_{\gamma}$$

Learn the weights from Geo's and Arushi's pairwise picks with a Bradley–Terry model, which models the probability that one item is preferred over another. Fit a weight vector $w_{p\gamma}$ for each person p, then combine the two (mean, or the minimum over p when both must like it):

$$P(n \succ_{p} m) = \sigma\!\big(w_{p\gamma}\,(s_{n\gamma} - s_{m\gamma})\big)$$

Until enough picks exist, start from hand-set weights. Alert when S_n clears a threshold set as a quantile of recent scores.

Build phases

Ship each phase end to end before starting the next. Geo reviews at every gate.

1. **Skeleton and config.** Python repo, SQLite store, one YAML config holding neighborhoods, hard filters, office addresses, departure window, constraint dimensions and starting weights.
   - Done when: `gem run --dry` loads the config and prints the planned fetches.
2. **Collector agent.** A headless agent run (Claude Agent SDK, or `claude -p` with only the web-fetch tool allowed) takes the search URLs from the config. It parses the cards, skips listings already in the database, and fetches the listing page for each new one. When the listing shows a "Floor plan" label, it also fetches `/media_gallery`. It writes a JSON record per listing. Deterministic code validates and stores that record. The agent never writes to the database directly.
   - Done when: one run over 3 neighborhoods stores every unique listing; 20 hand-checked records have at least 95% of fields correct; every listing with a "Floor plan" label has its plan URL.
3. **Floor-plan extraction.** Download plans from `photos.zillowstatic.com`, extract them to the schema, run the constraint checks.
   - Done when: on 30 hand-labelled plans (Parkline #22G is the first), room count is exact on at least 90% and printed dimensions are read within 1 inch on at least 95%.
4. **Commute.** r5py with GTFS and OpenStreetMap, travel times from each listing to both offices, cached per building.
   - Done when: 10 sample addresses land within ±5 minutes of a mainstream trip planner checked by hand.
5. **Price model.** Net effective rent normalization, then the hedonic fit with the CSV anchor.
   - Done when: residuals are computed for every stored listing, and out-of-sample error is reported once 300 or more listings exist.
6. **Score and alerts.** Hard filters, gem score, alert delivery. Each alert carries: net and base rent, a floor-plan thumbnail, room dimensions, both commute times, the score broken down by component, and the listing link. Alert once per listing, and again on a price drop.
   - Done when: a seeded test listing triggers exactly one alert with every field filled.
7. **Review UI and preference learning.** A small local web page listing candidates, where each of us can thumbs-up, thumbs-down, or pick between pairs. Refit the Bradley–Terry weights weekly.
   - Done when: picks are stored per person and a refit changes the ranking.

8. **Scheduling and ops.** Run 3 to 4 times a day (launchd on Geo's Mac, or a cloud scheduled task), with run logs, a failure alert, and a summary of fetch counts per run.
    - Done when: it runs for 7 days unattended with no missed runs.

Known pitfalls

All of these came up during prototyping.

- **Duplicate cards.** Featured and sponsored cards repeat regular ones. The PLG 2BR search returned 11 cards for 9 unique listings, and the page heading claimed 8. Deduplicate on the listing URL with its query string removed (`?featured=1`, `?infeed=1`).
- **Three kinds of price.** Cards show "base rent," "net effective" (with free months and lease length), or "total monthly price" (fees included). Normalize everything to net effective rent per month, including mandatory fees, before scoring.
- **Most listings have no plan.** Check for the "Floor plan" label on the listing page before fetching the gallery.
- **Virtual staging.** Photo furniture is often computer-rendered. Never use it for scale.
- **Square footage is unreliable.** It's often missing, and stated as approximate when present. Prefer the area computed from the plan; when both exist, store both and flag gaps over 10%.
- **Income-restricted units.** Housing-lottery units appear as ordinary listings with text like "specific income requirements" or "HPD." Detect that text and flag or exclude them, because their rents would distort the price model.
- **Unit-name variants.** The same unit can appear as `6A`, `6AW` or `6A-TERRACE`. Normalize unit IDs per building.
- **Noisy neighborhood medians.** The PLG 2BR median jumped from $3,400 to $4,100 between Aug and Sep 2026. Smooth with a 3-month mean.
- **Robots-disallowed URLs.** The fetch tool refuses `streeteasy.com/rental/<id>`. Build `/building/<slug>/<unit>` URLs instead.
- **Blocked shell fetches are expected.** Direct shell requests to StreetEasy may be blocked. That's the reason for the agent fetch path. Don't work around it.
- **OpenStreetMap was blocked in the cloud sandbox.** Download the extract on Geo's machine.

Open decisions for Geo

The builder should ask about each of these when it reaches the phase that needs it, not before.

- [ ] **Neighborhoods.** Current candidates: Williamsburg, Long Island City, Hunters Point, Greenpoint, Prospect Lefferts Gardens, Crown Heights, Prospect Heights, Park Slope, Fort Greene, South Harlem, Upper East Side. Which are in, and with what weights?
- [ ] **Hard filters.** Minimum beds and baths (2BR/2BA required?), maximum net effective rent, minimum sq ft.
- [ ] **Offices.** Exact address for Radical AI at the Brooklyn Navy Yard and for Arushi's office. Departure window. Max of the two commutes or a weighted sum?
- [ ] **Constraint dimensions.** Murphy bed size (queen or full) and the clear wall width and open depth it needs. Wall length for the desk and game setup. Minimum kitchen counter run.
- [ ] **Alert channel.** For two people: a push service (e.g. ntfy), a Telegram bot, or iMessage via the Mac.
- [ ] **Where it runs.** launchd on Geo's Mac (OpenStreetMap downloads work there) or a cloud scheduled task.
- [ ] **Run frequency and alert threshold.** Default: 3 runs a day, alert above the 90th percentile of recent scores.
- [ ] **Model budget.** Which models to use for the fetch agent and for plan extraction, and the monthly API spend cap.